

[SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS:

Total Marks: 50

Annexure A

Code of Conduct:

I DOMA SAKETH-192525018____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Debugging & Optimisation Task

Question 4: Optimization and Security Analysis of Diffie-Hellman Key Exchange

Aim

To debug a Python implementation of the Diffie-Hellman key exchange, optimize modular exponentiation for large prime values, and enhance security against Man-in-the-Middle (MITM) attacks.

A. Debug Incorrect Shared Key Generation Due to Improper Modular Arithmetic

Common Errors

Error 1: Incorrect Shared Key Formula

Incorrect Code

```
shared_key = (public_key ** private_key) % p
```

This becomes very slow for large values because the exponent is calculated first.

Correct Code

```
shared_key = pow(public_key, private_key, p)
```

The built-in `pow()` function performs fast modular exponentiation efficiently.

Error 2: Different Prime Numbers

Both users must use the **same prime number (p)** and **generator (g)**.

Incorrect:

Alice uses $p = 23$

Bob uses $p = 29$

Correct:

$p = 23$

$g = 5$

Error 3: Wrong Public Key Calculation

Incorrect:

$A = g * \text{private_key} \% p$

Correct:

$A = \text{pow}(g, \text{private_key}, p)$

Correct Python Program

```
# Diffie-Hellman Key Exchange
```

```
# Public values
```

```
p = 23    # Prime number
```

```
g = 5     # Primitive root
```

```
# Private keys
```

```
alice_private = 6
```

```
bob_private = 15
```

```
# Public keys
```

```
alice_public = pow(g, alice_private, p)
```

```
bob_public = pow(g, bob_private, p)
```

```
# Shared secret
```

```
alice_shared = pow(bob_public, alice_private, p)
```

```
bob_shared = pow(alice_public, bob_private, p)
```

```
print("Prime Number (p):", p)
```

```

print("Generator (g):", g)

print("\nAlice Public Key:", alice_public)
print("Bob Public Key:", bob_public)

print("\nAlice Shared Key:", alice_shared)
print("Bob Shared Key:", bob_shared)

if alice_shared == bob_shared:
    print("\nKey Exchange Successful")
else:
    print("\nKey Exchange Failed")

```

Sample Output

Prime Number (p): 23
Generator (g): 5

Alice Public Key: 8
Bob Public Key: 19

Alice Shared Key: 2
Bob Shared Key: 2

Key Exchange Successful

B. Optimize Exponentiation Operations for Large Prime Values

Optimization Techniques

1. Use Fast Modular Exponentiation

Instead of:

$A = (g^{**} \text{private_key}) \% p$

Use:

$A = \text{pow}(g, \text{private_key}, p)$

The pow() function uses the **Square-and-Multiply** algorithm, making exponentiation much faster for large numbers.

2. Reuse Public Parameters

Generate the prime number (p) and generator (g) once and reuse them instead of recalculating.

3. Avoid Unnecessary Variables

Compute public and shared keys directly using pow() to reduce memory usage.

C. Mitigate Man-in-the-Middle (MITM) Attacks

Problem

Basic Diffie-Hellman does **not authenticate** communicating parties.

An attacker can intercept the public keys and establish separate shared keys with Alice and Bob.

Alice ----> Attacker ----> Bob

Alice believes she is communicating with Bob, while Bob believes he is communicating with Alice.

Solution

Use Digital Signatures

- Alice signs her public key using her private signing key.
- Bob verifies the signature using Alice's public verification key.

Use Digital Certificates

A trusted Certificate Authority (CA) verifies the identity of each participant.

Use Authenticated Diffie-Hellman

Protocols such as **TLS** combine Diffie-Hellman with certificates to prevent MITM attacks.

Performance Impact

Enhancement	Performance Impact	Security Benefit
Fast Modular Exponentiation (pow())	Faster execution	Efficient computation
Digital Signatures	Slight increase in computation	Prevents impersonation
Digital Certificates	Small communication overhead	Authentication of users
Authenticated Diffie-Hellman	Moderate overhead	Protects against MITM attacks

Analysis

- The main debugging issue was incorrect modular arithmetic, which was fixed using Python's efficient pow(base, exponent, modulus) function.
- Performance was improved by using fast modular exponentiation and eliminating unnecessary computations.
- Security was strengthened by recommending authentication mechanisms such as digital signatures and certificates to prevent Man-in-the-Middle attacks.
- Although authentication introduces a small computational overhead, the security benefits greatly outweigh the performance cost.

Result

- Successfully debugged the Diffie-Hellman implementation by correcting modular arithmetic.
- Optimized exponentiation using Python's built-in pow() function for efficient large-number computations.
- Enhanced security by proposing digital signatures, certificates, and authenticated Diffie-Hellman to mitigate Man-in-the-Middle attacks.
- The optimized implementation is faster, more reliable, and suitable for secure key exchange in modern communication systems.

Console

Run

```
Run started
Initializing environment
Installing packages
Running code
Diffie-Hellman Key Exchange
-----
Prime Number (p): 23
Generator (g): 5

Alice's Private Key:
6
Bob's Private Key: 15

Alice's Public Key:
8
Bob's Public Key: 19

Alice's Shared Secret:
2
Bob's Shared Secret: 2

Key Exchange Successful!
Shared Secret Key: 2
Run completed in 10073.800000011921ms
```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
		Optimizes			

Optimization	20%	code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

